

Building a Multi-Host, Multi-Container Orchestration and Distributed System using Docker

This article is about the next generation Docker clustering and distributed system, which comes with various interesting built-in features like orchestration, self-healing, self-organising, resilience and security.



Docker is an open platform designed to help both developers and systems administrators to build, ship and run distributed applications by using containers. Docker allows the developer to package an application with all the parts it needs, such as libraries and other dependencies, and to ship it all out as one package. This ensures that the application will run on any other Linux machine, regardless of any customised settings that the machine might have, which could differ from the machine used for writing and testing the code.

Docker Engine 1.12 can be rightly called the next generation Docker clustering and distributed system. One of the major highlights of this release is the Docker Swarm Mode, which provides a powerful yet optional ability to create coordinated groups of decentralised Docker engines. Swarm Mode combines your engine in swarms of any scale. It is self-organising and self-healing. It enables an infrastructure-agnostic topology. The newer version democratises orchestration with out-of-the-box capabilities for multi-container on multi-host app deployments, as shown in Figure 1.

Built as a uniform building block for self-organising and healing a group of engines, Docker ensures that orchestration is accessible to every developer and operation user. The new Swarm Mode adopts the de-centralised architecture rather than the centralised one (key-value store) as seen in the earlier Swarm

releases. Swarm Mode uses the Raft consensus algorithm to perform leader selection, and maintain the cluster's states.

In the Swarm Mode, all Docker engines will unite into a cluster with a management tier. It is basically a master-slave system, but all Docker engines will be united and they will maintain a cluster state. Instead of running a single container, you declare a desired state for your application, which means multiple containers and then the engines themselves will maintain that state. Additionally, a new 'Docker service' feature has been added in the new release. The command, '*docker service create*' is expected to be an evolution of '*docker run*', which is an imperative command and it helps you to get the container up and running. The new *docker service create* command declares that you have to set up a server, which can run one or more containers and those containers will run, provided the state you declare for the service is maintained in the engine inside the distributed store based on the Raft consensus protocol. That brings up the desired state reconciliation. Whenever any node in the cluster goes down, the Swarm itself will recognise that there has been a deviation in the desired state, and it will bring up a new instance to reconstruct the reconciliation.

Docker Swarm Mode is used to orchestrate distributed systems at any scale. It includes primitives for node